Logistic Regression for Prediction, ROC Analysis, and Model Evaluation

RSH 7033 – Applied Multivariate Statistics

1. Introduction

In many statistical applications, the central objective is to model the relationship between a response variable and a set of explanatory variables. In classical regression analysis, this relationship is typically expressed through a linear model where the response variable is assumed to be continuous and normally distributed.

However, in many real-world situations, the response variable is binary. For example, in the placekicking dataset:

$$Y_r \in \{0, 1\}$$

where:

- $Y_r = 1$ denotes success
- $Y_r = 0$ denotes failure

Thus, we model:

$$P(Y_r = 1 \mid X_r)$$

Logistic regression allows us to model probabilities directly and provides interpretable parameter estimates.

2. Logistic Regression for Prediction

The primary purpose of logistic regression, within the context of classification, is to determine whether an observation belongs to one of two populations. This decision is not made directly from the model coefficients, but rather through the estimated probability of success.

For each observation, the fitted model produces:

$$\hat{\pi}_r = \frac{e^{\hat{\beta}_0 + \hat{\beta}_1 x_{r1} + \cdots + \hat{\beta}_p x_{rp}}}{1 + e^{\hat{\beta}_0 + \hat{\beta}_1 x_{r1} + \cdots + \hat{\beta}_p x_{rp}}}$$

This value represents the estimated probability that the observation belongs to the class $Y = 1$.

2.1. From Probability to Classification

In order to make decisions, probabilities must be converted into class labels. A commonly used rule is:

- If $\hat{\pi}_r > 0.5$, classify as $Y = 1$
- Otherwise, classify as $Y = 0$

While this rule is intuitive, it is important to recognize that **0.5 is not universally optimal**. The choice of threshold directly affects classification performance.

> **At the graduate level, students must clearly understand:**
>
> Logistic regression does not classify observations — it produces probabilities.
>
> Classification is a separate decision layer imposed afterward.

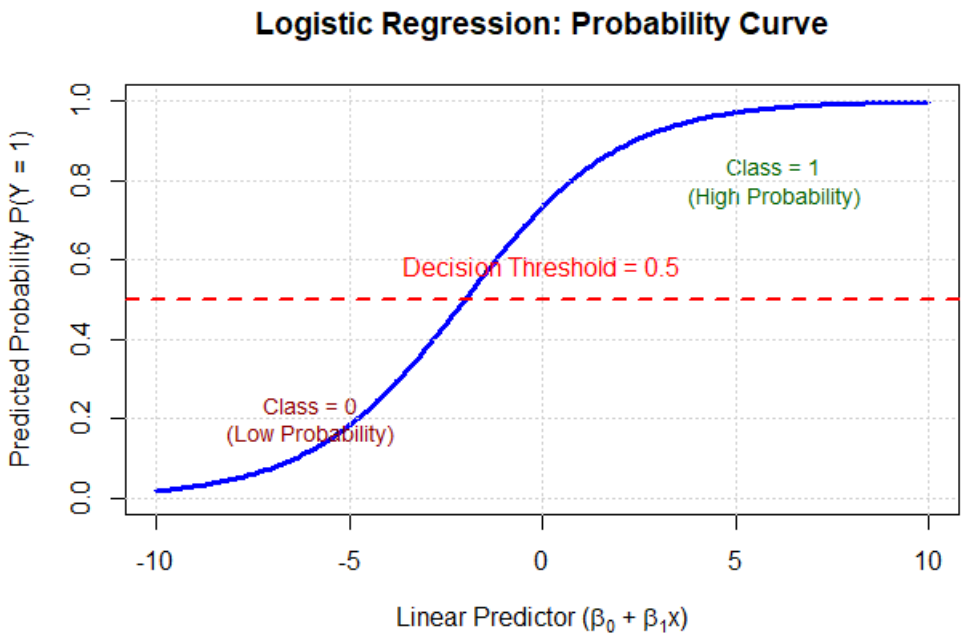

Figure 1: **Logistic Curve Visualization (Concept Builder).** The figure displays the logistic regression probability curve obtained from R output. The horizontal axis represents the linear predictor ($\beta_0 + \beta_1 x$), while the vertical axis shows the predicted probability of success $P(Y = 1)$. The S-shaped curve illustrates how probabilities increase nonlinearly as the predictor increases. The dashed red line at 0.5 indicates a commonly used decision threshold for classification. Observations with predicted probabilities above this threshold are classified as the positive class (Class = 1), while those below are classified as the negative class (Class = 0). This visualization highlights that logistic regression produces probabilities, and classification decisions depend on a chosen threshold.

3. Evaluating Classification: Sensitivity and Specificity

To understand classification performance, we define two fundamental measures.

3.1. Sensitivity

Sensitivity is the probability that a truly positive observation is correctly classified:

$$P(\text{classify positive} \mid \text{actual positive})$$

3.2. Specificity

Specificity is the probability that a truly negative observation is correctly classified:

$$P(\text{classify negative} \mid \text{actual negative})$$

3.3. Conceptual Understanding

- Sensitivity focuses on detecting successes
- Specificity focuses on detecting failures

In many applied settings, improving one often worsens the other.

4. False Rates and Trade-Off

We define:

$$\text{False Positive Rate} = 1 - \text{Specificity}$$

$$\text{False Negative Rate} = 1 - \text{Sensitivity}$$

4.1. Interpretation

Increasing sensitivity typically increases false positives.

Increasing specificity typically increases false negatives.

This trade-off is central to classification problems and motivates the use of ROC curves.

5. Confusion Matrix (Foundation of Evaluation)

5.1. Structure

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

5.2. Placekicking Example

```
            Predicted
Actual       0    1
      0     23   15
      1     29  215
```

Step-by-step:

- TN = 23
- FP = 15
- FN = 29
- TP = 215

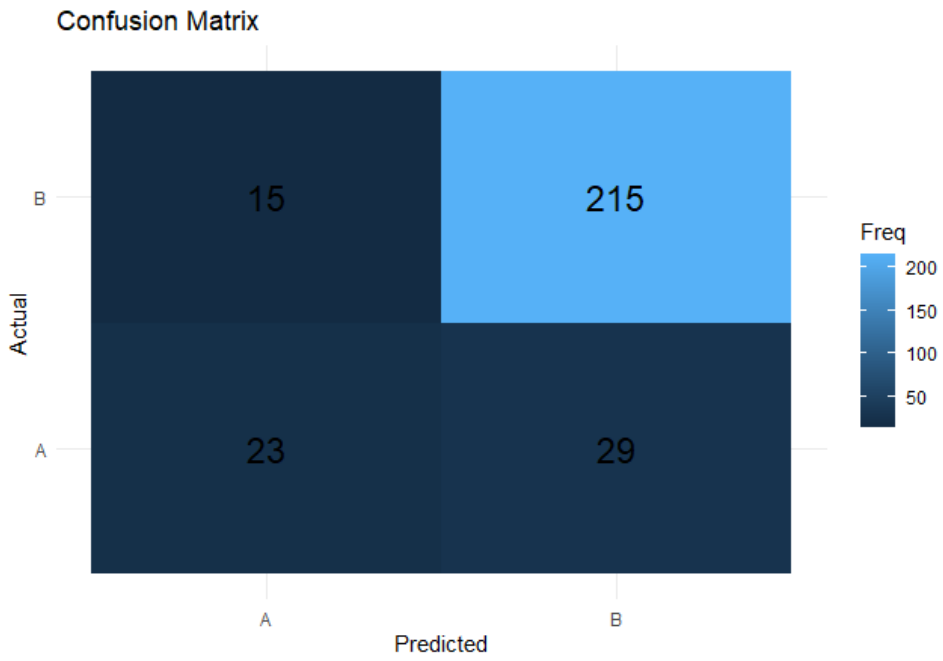

Figure 2: **Model Classification Performance: Confusion Matrix Heatmap.** This figure presents the confusion matrix for the logistic regression model in a heatmap format. The rows represent the actual class labels, while the columns represent the predicted class labels. Each cell displays the frequency of observations corresponding to true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP). The color intensity reflects the magnitude of classification counts, providing a visual assessment of model performance. A higher concentration of values along the diagonal indicates strong classification accuracy, while off-diagonal values highlight misclassification errors.

5.3. Metrics

$$\text{Sensitivity} = \frac{215}{215 + 29} = 0.8811$$

$$\text{Specificity} = \frac{23}{23 + 15} = 0.6053$$

$$1 - \text{Specificity} = 0.3947$$

6. Placekicking Example — Full R Output

6.1. Model Estimation

```
> mod.fit <- glm(
+ good ~ week + distance + change + elap30 + PAT + type +
   field + wind,
+ data = placekick,
+ family = binomial(link = "logit")
+ )
> print(mod.fit)

Call: glm(formula = good ~ week + distance + change + elap30
    + PAT +
   type + field + wind, family = binomial(link = "logit"),
      data = placekick)

Coefficients:
(Intercept) week distance change elap30 PAT
  4.781039 -0.024774 -0.085899 -0.349698 0.004795 1.245801
      type field wind
  0.309970 -0.216276 -0.630237

Degrees of Freedom: 1424 Total (i.e. Null); 1416 Residual
Null Deviance: 1013
Residual Deviance: 753.5 AIC: 771.5

> summary(mod.fit)

Call:
glm(formula = good ~ week + distance + change + elap30 + PAT
    +
   type + field + wind, family = binomial(link = "logit"),
      data = placekick)

Coefficients:
          Estimate Std. Error z value Pr(>|z|)
(Intercept) 4.781039 0.558442 8.561 < 2e-16 ***
week -0.024774 0.019528 -1.269 0.20456
distance -0.085899 0.011228 -7.650 2e-14 ***
change -0.349698 0.195358 -1.790 0.07345 .
elap30 0.004795 0.010552 0.454 0.64955
PAT 1.245801 0.385085 3.235 0.00122 **
type 0.309970 0.279175 1.110 0.26687
field -0.216276 0.254247 -0.851 0.39496
wind -0.630237 0.343864 -1.833 0.06683 .
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' '
   1

(Dispersion parameter for binomial family taken to be 1)

   Null deviance: 1013.43 on 1424 degrees of freedom
```

```
Residual deviance: 753.52 on 1416 degrees of freedom
AIC: 771.52

Number of Fisher Scoring iterations: 6

> Anova(mod.fit)
Analysis of Deviance Table (Type II tests)

Response: good
       LR Chisq Df Pr(>Chisq)
week 1.617 1 0.2035650
distance 69.122 1 < 2.2e-16 ***
change 3.175 1 0.0747900 .
elap30 0.207 1 0.6493575
PAT 11.117 1 0.0008554 ***
type 1.247 1 0.2640669
field 0.734 1 0.3915089
wind 3.224 1 0.0725444 .
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' '
   1
```

6.2. Predicted Probabilities

```
> pi.hat <- predict(mod.fit, type = "response")
> head(pi.hat)
      1 2 3 4 5 6
0.9434752 0.9577942 0.9876288 0.9248228 0.9877382 0.9366464

> pi.hat.v <- predict(mod.fit, newdata = valid, type = "
   response")
> head(pi.hat.v)
      1 2 3 4 5 6
0.9433265 0.6635789 0.4760922 0.9877546 0.9866572 0.9849546
```

6.3. Classification (Cut-off = 0.5)

```
> cutoff <- 0.5
>
> pred.good <- ifelse(pi.hat > cutoff, 1, 0)
> pred.good.v <- ifelse(pi.hat.v > cutoff, 1, 0)
>
> # Training
> res.train <- summarize.class(placekick[,9], pred.good)
```

```
--- Confusion Matrix ---
      classify
original 0 1
     0 18 145
     1 13 1249

--- Row Proportions ---
      classify
original 0 1
     0 0.1104 0.8896
     1 0.0103 0.9897

--- Overall Accuracy ---
[1] 0.8891
>
> # Validation
> res.valid <- summarize.class(valid[,9], pred.good.v)

--- Confusion Matrix ---
      classify
original 0 1
     0 3 35
     1 8 236

--- Row Proportions ---
      classify
original 0 1
     0 0.0789 0.9211
     1 0.0328 0.9672

--- Overall Accuracy ---
[1] 0.8475
```

6.4. Confusion Matrix Heatmap

```
> # Confusion Matrix Heatmap
> cm <- table(placekick[,9], pred.good)
> cm_df <- as.data.frame(cm)
>
> ggplot(cm_df, aes(Var2, Var1, fill = Freq)) +
+ geom_tile() +
+ geom_text(aes(label = Freq), size = 6) +
+ labs(title = "Confusion Matrix Heatmap",
+ x = "Predicted",
+ y = "Actual") +
+ theme_minimal()
>
```

```
> cm
   pred.good
      0  1
  0  18 145
  1  13 1249
```

6.5. Accuracy Measures

```
> prop.table(table(placekick$good, pred.good), 1)

     pred.good
          0      1
0    0.1104 0.8896
1    0.0103 0.9897

> overall accuracy
[1] 0.8891
```

Interpretation (DETAILED)

- Model is extremely good at predicting successes (1)
- But performs poorly for failures (0)
- This indicates class imbalance bias

6.6. Full R Script (Placekicking Example)

```
##############################################################
# RSH 7033 -- Logistic Regression Full Script
# Placekicking Example (Final Version)
##############################################################

# ------------------------------------------------------------
# 1. Load Required Libraries
# ------------------------------------------------------------
library(ROCR)
library(car)
library(stats)
library(graphics)

# ------------------------------------------------------------
# 2. Load Data
# ------------------------------------------------------------
```

```r
placekick <- read.csv("Placekick.csv")
valid <- read.csv("valid.csv")

# ---------------------------------------------------------
# 3. Fit Logistic Regression Model
# ---------------------------------------------------------
mod.fit <- glm(
  formula = good ~ week + distance + change + elap30 + PAT +
     type + field + wind,
  data = placekick,
  family = binomial(link = "logit")
)

# ---------------------------------------------------------
# 4. Model Output
# ---------------------------------------------------------
cat("\n===== MODEL SUMMARY =====\n")
print(mod.fit)

cat("\n===== DETAILED SUMMARY =====\n")
summary(mod.fit)

cat("\n===== ANOVA (Type II) =====\n")
Anova(mod.fit)

# ---------------------------------------------------------
# 5. Predicted Probabilities
# ---------------------------------------------------------
cat("\n===== TRAINING PROBABILITIES =====\n")
pi.hat <- predict(mod.fit, type = "response")
print(head(pi.hat))

cat("\n===== VALIDATION PROBABILITIES =====\n")
pi.hat.v <- predict(mod.fit, newdata = valid, type = "
   response")
print(head(pi.hat.v))

# ---------------------------------------------------------
# 6. Classification Function
# ---------------------------------------------------------
summarize.class <- function(original, classify) {
  class.table <- table(original, classify)
  numb <- rowSums(class.table)
  prop <- round(class.table / numb, 4)
  overall <- round(sum(diag(class.table)) / sum(class.table),
      4)

  cat("\n--- Confusion Matrix ---\n")
  print(class.table)
```

```r
  cat("\n--- Row Proportions ---\n")
  print(prop)

  cat("\n--- Overall Accuracy ---\n")
  print(overall)

  return(list(class.table = class.table,
              prop = prop,
              overall.correct = overall))
}

# ------------------------------------------------------------
# 7. Classification (Cutoff = 0.5)
# ------------------------------------------------------------
cat("\n===== CLASSIFICATION: CUTOFF = 0.5 =====\n")

cutoff <- 0.5

pred.good <- ifelse(pi.hat > cutoff, 1, 0)
pred.good.v <- ifelse(pi.hat.v > cutoff, 1, 0)

cat("\nTraining Data:\n")
res.train <- summarize.class(placekick[,9], pred.good)

cat("\nValidation Data:\n")
res.valid <- summarize.class(valid[,9], pred.good.v)

# ------------------------------------------------------------
# 8. Classification (Cutoff = 0.7)
# ------------------------------------------------------------
cat("\n===== CLASSIFICATION: CUTOFF = 0.7 =====\n")

cutoff <- 0.7

pred.good <- ifelse(pi.hat > cutoff, 1, 0)
pred.good.v <- ifelse(pi.hat.v > cutoff, 1, 0)

cat("\nTraining Data:\n")
summarize.class(placekick[,9], pred.good)

cat("\nValidation Data:\n")
summarize.class(valid[,9], pred.good.v)

# ------------------------------------------------------------
# 9. ROC Curve
# ------------------------------------------------------------
cat("\n===== ROC CURVE =====\n")

pred <- prediction(pi.hat, placekick[,9])
perf <- performance(pred, "sens", "fpr")
```

```r
# Plot ROC
plot(perf, main = "ROC Curve")
abline(a = 0, b = 1)

# AUC
auc <- performance(pred, "auc")
cat("\nAUC:\n")
print(auc@y.values)

# ------------------------------------------------------------
# 10. Logistic Function Plots (PiPlot)
# ------------------------------------------------------------
cat("\n===== LOGISTIC FUNCTION PLOTS =====\n")

par(mfrow = c(1,2))

beta0 <- 1
beta1 <- 0.5

curve(exp(beta0+beta1*x)/(1+exp(beta0+beta1*x)),
     xlim = c(-15,15),
     main = "beta1 = 0.5",
     xlab = "x",
     ylab = "pi")

beta1 <- -0.5

curve(exp(beta0+beta1*x)/(1+exp(beta0+beta1*x)),
     xlim = c(-15,15),
     main = "beta1 = -0.5",
     xlab = "x",
     ylab = "pi")

# ------------------------------------------------------------
# 11. Cross-Validation (Leave-One-Out)
# ------------------------------------------------------------
cat("\n===== CROSS VALIDATION =====\n")

cv <- function(model, data.set) {
  N <- nrow(data.set)
  pi.hat.cv <- numeric(length = N)

  for (r in 1:N) {
    mod.fit.cv <- glm(
      formula = model,
      data = data.set[-r,],
      family = binomial(link = "logit")
    )
```

```r
    pi.hat.cv[r] <- predict(
      mod.fit.cv,
      newdata = data.set[r,],
      type = "response"
    )
  }

  return(pi.hat.cv)
}

# Run CV
pi.hat.cv <- cv(
  good ~ week + distance + change + elap30 + PAT + type +
     field + wind,
  placekick
)

pred.cv <- ifelse(pi.hat.cv > 0.5, 1, 0)

cat("\nCross-Validation Results:\n")
summarize.class(placekick[,9], pred.cv)

# ----------------------------------------------------------
# 12. PCA Visualization
# ----------------------------------------------------------
cat("\n===== PCA VISUALIZATION =====\n")

# Use numeric predictors only
X <- placekick[, c("week","distance","change","elap30","PAT",
   "type","field","wind")]

pca <- prcomp(X, scale = TRUE)

pc <- pca$x[,1:2]

plot(pc,
    col = ifelse(pred.good == 1, "red", "black"),
    pch = 16,
    main = "PCA Classification Plot",
    xlab = "PC1",
    ylab = "PC2")

legend("topright",
      legend = c("Predicted Success","Predicted Failure"),
      col = c("red","black"),
      pch = 16)

##############################################################
# END OF SCRIPT
##############################################################
```

Interpretation:

The model strongly favors predicting success (class imbalance effect).

7. Receiver Operating Characteristic (ROC) Curve

A receiver operating characteristic (ROC) curve is a graphical tool used to evaluate the performance of a classification model across a range of classification thresholds.

In the context of logistic regression, the ROC curve is constructed by examining how the classification results change as the cut-off probability varies.

7.1. Definition

The ROC curve plots:

Sensitivity (True Positive Rate) vs. $1-$Specificity (False Positive Rate)

for a sequence of cut-off values.

7.2. Purpose

The ROC curve allows the researcher to:

- Evaluate classification performance across all possible thresholds
- Understand the trade-off between sensitivity and false positive rate
- Select a cut-off probability appropriate for the specific application

7.3. How the ROC Curve is Constructed

To construct the ROC curve, the following steps are performed:

1. A sequence of cut-off probabilities is selected (e.g., from 0 to 1).
2. For each cut-off value:

- Classify observations based on the rule $\hat{\pi} >$ cut-off
- Compute:
  - Sensitivity
  - False positive rate

3. Plot sensitivity versus false positive rate.

7.4. Interpreting the Axes

- The x-axis represents the false positive rate: $1 -$ specificity
- The y-axis represents sensitivity: $P(\text{classify positive} \mid \text{actual positive})$

7.5. Reference Line (Chance Line)

A diagonal line from $(0, 0)$ to $(1, 1)$ represents random classification.

- Any model that performs better than random will have a curve above this line
- A model performing worse than random would lie below this line

7.6. Interpretation of the Curve

A good classification model will exhibit:

- High sensitivity (near 1)
- Low false positive rate (near 0)

Thus, the ROC curve should be pushed toward the upper-left corner.

7.7. Area Under the Curve (AUC)

A commonly used summary measure of the ROC curve is the area under the curve (AUC).

- AUC = 0.5 $\rightarrow$ no predictive ability (random)
- AUC close to 1 $\rightarrow$ excellent classification performance

7.8. Key Result

The larger the area under the ROC curve, the better the classification procedure.

7.9. Selecting an Appropriate Cut-Off

The ROC curve provides a systematic way to select a cut-off probability:

- Identify a point on the curve that balances:
  - sensitivity
  - false positive rate

The choice depends on the context of the problem.

7.10. Instructor Insight (DBA-Level Emphasis)

There is no universally optimal cut-off value.
Instead, the selection must reflect:

- the cost of misclassification
- the importance of detecting true positives
- the tolerance for false positives

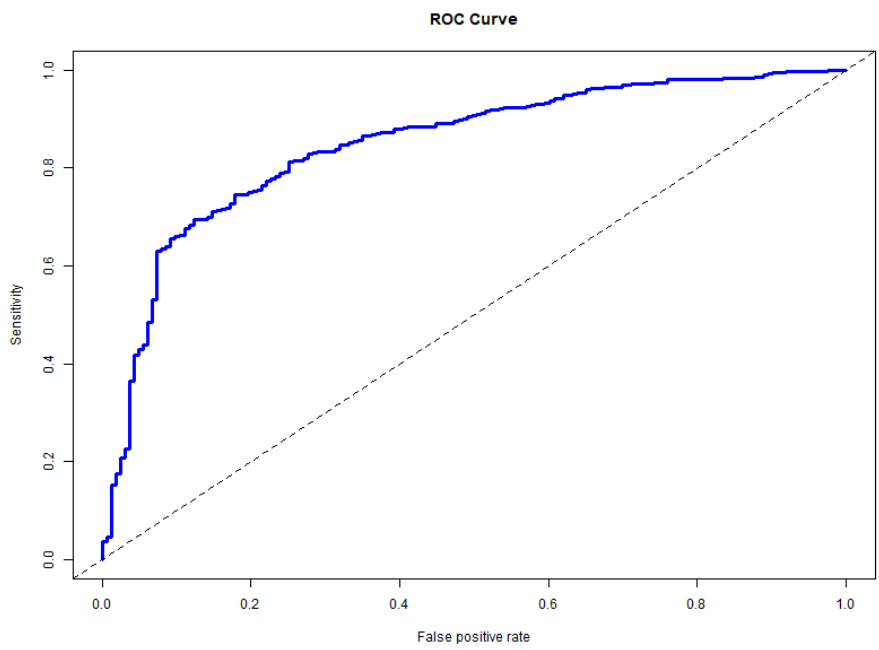

Figure 3: **ROC Curve.** The ROC curve plots sensitivity against the false positive rate across thresholds. A curve closer to the upper-left corner indicates better classification performance.

8. R Implementation of ROC

8.1. Application: Placekicking Data

We now construct the ROC curve using the predicted probabilities from the logistic regression model.

8.2. R Code

```
library(ROCR)

save.pred <- prediction(predictions = pi.hat, labels =
   placekick[,9])

save.perf <- performance(prediction.obj = save.pred,
                    measure = "sens",
                    x.measure = "fpr")

plot(save.perf, main = "ROC curve for placekicking data")
grid()
```

8.3. Explanation

The `prediction()` function combines the predicted probabilities and the true class labels. The `performance()` function then computes sensitivity and false positive rate for a sequence of cut-off values.

The resulting plot shows how classification performance changes across thresholds.

9. Extracting Cut-off Information

9.1. Reading Values from the ROC Curve

In some situations, we may wish to determine:

- the sensitivity corresponding to a specific false positive rate
- the cut-off probability associated with a point on the curve

These values are stored within the ROC object.

9.2. Example

```
one.minus.spec <- save.perf@x.values[[1]]
sens <- save.perf@y.values[[1]]
cutoff <- save.perf@alpha.values[[1]]

ck <- one.minus.spec > 0.495 & one.minus.spec < 0.505

data.frame(
  false_positive_rate = one.minus.spec[ck],
  sensitivity = sens[ck],
  cutoff = cutoff[ck]
)
```

9.3. Interpretation

- Each cut-off corresponds to:
  - a sensitivity value
  - a false positive rate

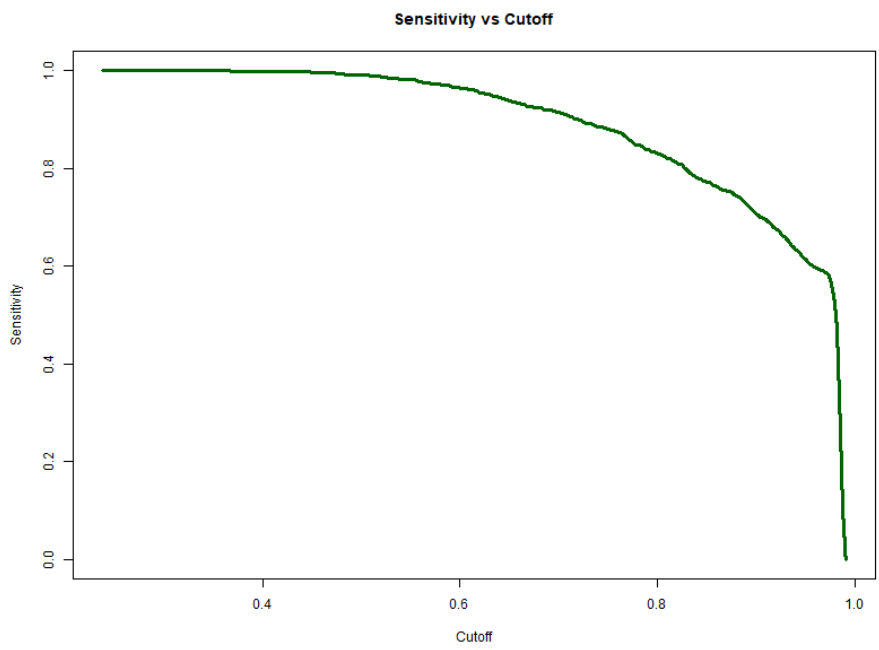

Figure 4: **Sensitivity vs Cut-off.** Sensitivity decreases as the classification threshold increases, illustrating the trade-off between detecting positives and model strictness.

10. Sensitivity vs Cut-off

```
save.perf2 <- performance(save.pred, "sens", "cutoff")
plot(save.perf2, main = "Sensitivity vs Cutoff")
```

Interpretation

- As cut-off increases:
  - Sensitivity decreases
  - Model becomes stricter

11. Area Under the Curve (AUC)

```
> performance(save.pred, "auc")@y.values
[[1]]
[1] 0.8471411
```

Meaning

- AUC = 0.847 $\rightarrow$ strong model

11.1. AUC Range

AUC Range	Meaning
0.5	Random
0.7–0.8	Acceptable
0.8–0.9	Strong
> 0.9	Excellent

12. Validation ROC

12.1. R Code (Validation ROC + Save Plot)

```
save.pred.v <- prediction(pi.hat.v, valid[,9])
save.perf.v <- performance(save.pred.v, "sens", "fpr")

png("roc_validation.png", width = 800, height = 600)

plot(save.perf.v,
    main = "ROC Curve (Validation Data)",
    col = "blue",
    lwd = 3)

abline(0, 1, lty = 2, col = "gray")
grid()

dev.off()

# AUC
performance(save.pred.v, "auc")@y.values
```

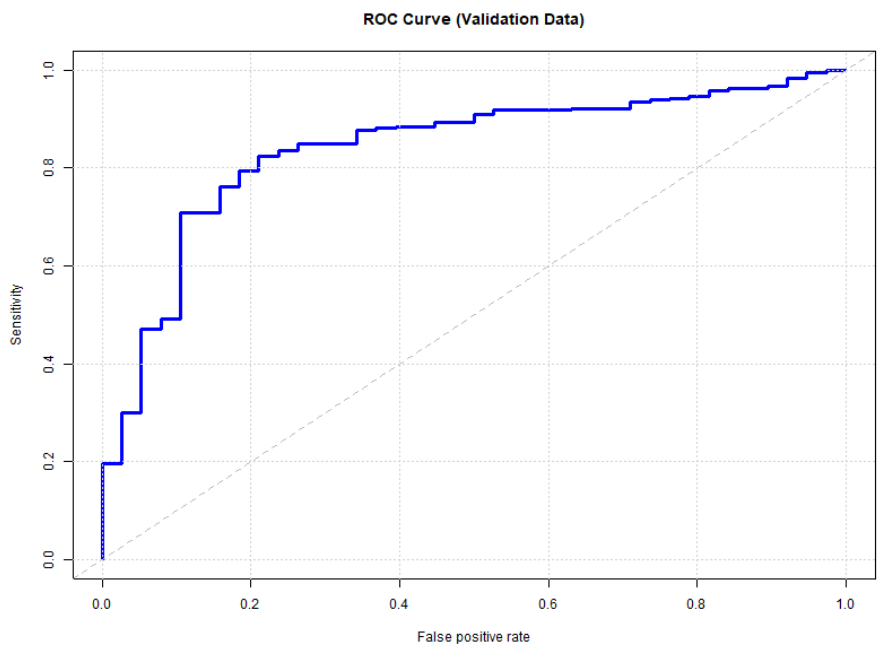

Figure 5: **ROC Curve (Validation Data).** This figure evaluates classification performance on unseen data. The ROC curve remains close to the upper-left corner, indicating strong predictive performance. The slight reduction in AUC compared to training data reflects expected generalization behavior rather than overfitting.

12.2. AUC (Validation)

```
> performance(save.pred.v, "auc")@y.values
[[1]]
[1] 0.839409
```

Interpretation

- Slight decrease → expected
- Indicates good generalization

13. Model Comparison (Critical Insight)

13.1. R Code

```
auc_full <- performance(pred_full, "auc")@y.values[[1]]
auc_dist <- performance(pred_dist, "auc")@y.values[[1]]
auc_elap <- performance(pred.elap, "auc")@y.values[[1]]
```

```r
cat("AUC (All variables) =", round(auc_full, 3), "\n")
cat("AUC (Distance only) =", round(auc_dist, 3), "\n")
cat("AUC (Elap30 only) =", round(auc_elap, 3), "\n")
```

13.2. R Output

```
AUC (All variables) = 0.847
AUC (Distance only) = 0.838
AUC (Elap30 only) = 0.56
```

Interpretation

- Distance alone $\approx$ full model $\rightarrow$ dominant predictor
- Elap30 $\rightarrow$ weak predictor
- Significance $\neq$ predictive power

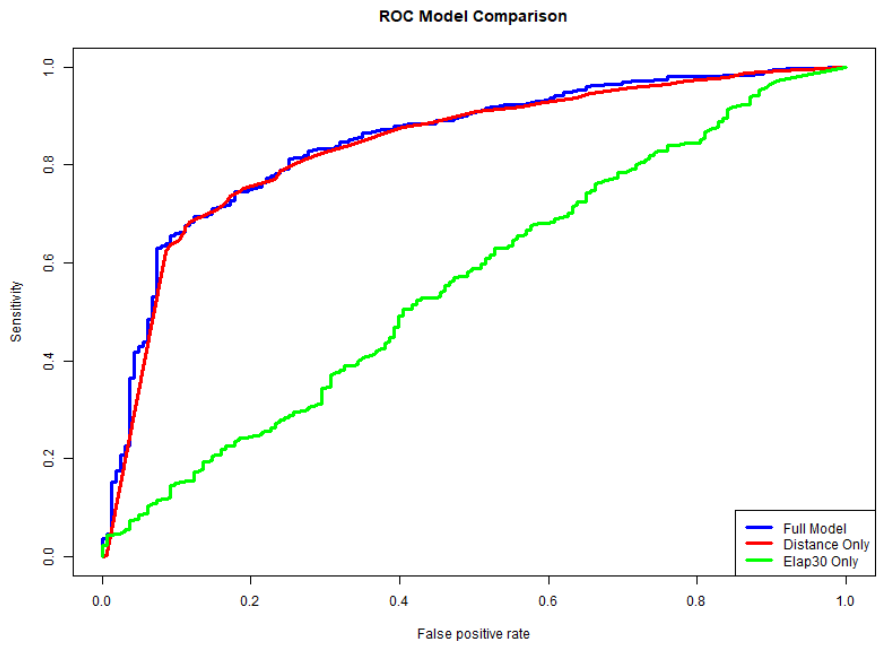

Figure 6: **ROC Curve Comparison Across Models.** The full model and distance-only model exhibit similar performance, indicating that distance is a dominant predictor. In contrast, the elap30-only model performs poorly, demonstrating weak predictive capability. This highlights that statistical significance does not necessarily imply predictive strength.

14. Complete Separation (Critical Theory)

Occurs when:

- Predictor perfectly separates classes

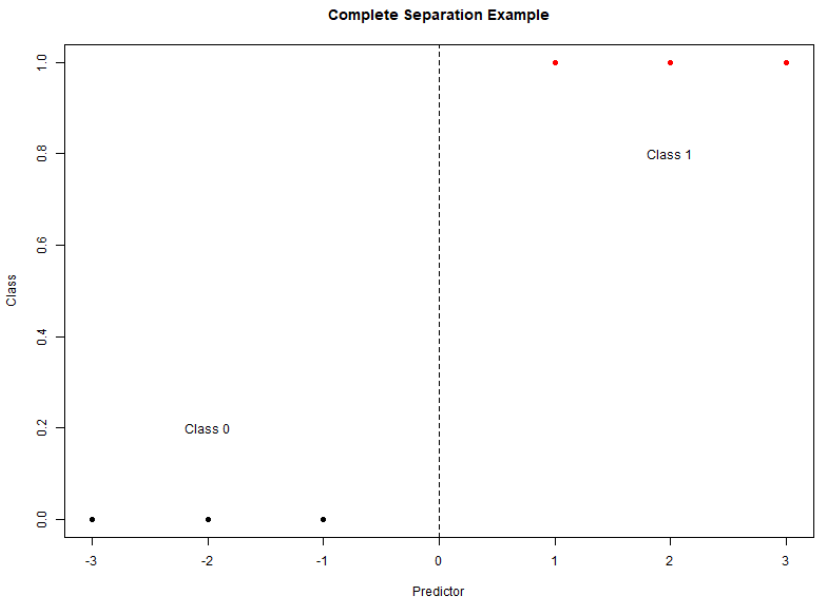

Figure 7: **Complete Separation in Logistic Regression.** This figure illustrates a scenario where a predictor perfectly separates classes. In such cases, logistic regression coefficients diverge toward infinity, leading to unstable model estimates and convergence warnings.

14.1. Meaning

- Estimates go to $\pm\infty$
- Model becomes unstable

14.2. Why It Happens

- Perfect classification boundary
- No overlap between groups

Figure 8: **Complete Separation in Logistic Regression.** This figure illustrates a scenario where a predictor perfectly separates classes. In such cases, logistic regression coefficients diverge toward infinity, leading to unstable model estimates and convergence warnings.

15. Practical Solutions

- Penalized logistic regression
- Exact logistic regression
- Remove problematic predictors

16. Final Interpretation Framework

When evaluating a logistic model:

- Step 1: Examine the confusion matrix to understand classification behavior.
- Step 2: Analyze the ROC curve to evaluate trade-offs between sensitivity and false positives.
- Step 3: Use AUC to compare competing models.
- Step 4: Select an appropriate cut-off based on application context.

Key Insight: There is no universally optimal cut-off. The choice depends on the cost of misclassification and domain-specific priorities.

17. DBA-Level Insight (VERY IMPORTANT)

There is NO universally optimal cut-off.

Instead:

- Healthcare $\rightarrow$ maximize sensitivity
- Finance $\rightarrow$ balance risk
- Operations $\rightarrow$ minimize false positives